

Como se aprecia, se decide probar cada factor controlable en tres niveles y variar cada factor de ruido en dos niveles. Aunque no es posible controlar los factores de ruido durante la producción, sí hay manera de hacerlo para fines experimentales. Estos datos representan la fuerza necesaria para desunir el ensamble, el cual se busca maximizar. ¿Según el Texto el arreglo escogido fue:

Creación del DF

```
import numpy as np
import pandas as pd

# 1. Definición de los datos
# Los datos de la tabla representan la fuerza de desunión (a maximizar)
# Las 9 filas son las combinaciones del Arreglo Interno (L9).
# Las 8 columnas son las repeticiones del Arreglo Externo (combinaciones de ruido).

datos = np.array([
    [19.1, 20.0, 19.6, 19.6, 19.9, 16.9, 9.5, 15.6], # Fila 1: A1 B1 C1 D1
    [21.9, 24.2, 19.8, 19.7, 19.6, 19.4, 16.2, 15.0], # Fila 2: A1 B2 C2 D4
    [20.4, 23.3, 18.2, 22.6, 15.6, 19.1, 16.7, 16.3], # Fila 3: A1 B3 C3 D2
    [24.7, 23.2, 18.9, 21.0, 18.6, 18.9, 17.4, 18.3], # Fila 4: A2 B1 C2 D3
    [25.3, 27.5, 21.4, 25.6, 25.1, 19.4, 18.6, 19.7], # Fila 5: A2 B2 C3 D1
    [24.7, 22.5, 19.6, 14.7, 19.8, 20.0, 16.3, 16.2], # Fila 6: A2 B3 C1 D2
    [21.6, 24.6, 18.6, 16.8, 23.6, 18.4, 19.1, 16.4], # Fila 7: A3 B1 C3 D2
    [24.4, 23.2, 19.6, 17.8, 16.8, 15.1, 15.6, 14.2], # Fila 8: A3 B2 C1 D3
    [28.6, 22.6, 22.7, 23.1, 17.3, 19.3, 19.9, 16.1] # Fila 9: A3 B3 C2 D4
])

# 2. Asignación de niveles para el Arreglo Ortogonal L9
# Las combinaciones del L9 para 4 factores (A, B, C, D) con 3 niveles (1, 2, 3)
# Típicamente se usa la estructura estándar L9 para 4 factores con 3 niveles:
niveles = pd.DataFrame({
    'A': [1, 1, 1, 2, 2, 2, 3, 3, 3],
    'B': [1, 2, 3, 1, 2, 3, 1, 2, 3],
    'C': [1, 2, 3, 2, 3, 1, 3, 1, 2],
    'D': [1, 2, 3, 3, 1, 2, 2, 3, 1]
})
```

Análisis Taguchi

```
# 1. Función para calcular la Razón Señal/Ruido (S/N)
# Para la característica "Mientras más grande, mejor" (maximizar la fuerza)
def calcular_sn(y):
    """Calcula la S/N para 'Mientras más grande, mejor' (Larger is Better)."""
    # Fórmula:  $SN = -10 * \log_{10}((1/n) * \sum(1/y_i^2))$ 
    n = len(y)
    termino_sum = np.sum(1 / y**2)
    return -10 * np.log10(termino_sum / n)

# 2. Cálculo de Media, Desviación Estándar y S/N para cada fila
niveles['Media'] = np.mean(datos, axis=1)
niveles['S'] = np.std(datos, axis=1, ddof=1) # Usar ddof=1 para desviación estándar muestral
niveles['SN'] = [calcular_sn(fila) for fila in datos]
```

```

#print("## Tabla de Resultados del Arreglo Interno")
#print(niveles.round(3))
#print("-" * 50)

# 3. Análisis de medias para la Razón Señal/Ruido (S/N)
# El objetivo es maximizar S/N para MINIMIZAR la variabilidad
analisis_sn = {}
for factor in ['A', 'B', 'C', 'D']:
    medias_sn_por_nivel = niveles.groupby(factor)['SN'].mean().reset_index()
    analisis_sn[factor] = medias_sn_por_nivel

niveles_optimos = {}
for factor, df in analisis_sn.items():
    # Encontrar el nivel que tiene el máximo S/N (mínima variabilidad)
    mejor_nivel_row = df.loc[df['SN'].idxmax()]
    nivel_optimo = int(mejor_nivel_row[factor])
    sn_max = round(mejor_nivel_row['SN'], 3)
    niveles_optimos[factor] = nivel_optimo

    print(f"Factor {factor}:")
    print(df.round(3))
    print(f"-> Nivel óptimo para S/N: {nivel_optimo} (S/N promedio: {sn_max} dB)")

## Factor A:
##      A      SN
## 0  1  24.961
## 1  2  26.046
## 2  3  25.568
## -> Nivel óptimo para S/N: 2 (S/N promedio: 26.046 dB)
## Factor B:
##      B      SN
## 0  1  25.216
## 1  2  25.754
## 2  3  25.604
## -> Nivel óptimo para S/N: 2 (S/N promedio: 25.754 dB)
## Factor C:
##      C      SN
## 0  1  24.728
## 1  2  25.859
## 2  3  25.987
## -> Nivel óptimo para S/N: 3 (S/N promedio: 25.987 dB)
## Factor D:
##      D      SN
## 0  1  25.695
## 1  2  25.522
## 2  3  25.357
## -> Nivel óptimo para S/N: 1 (S/N promedio: 25.695 dB)

# Niveles que Minimizan la Variabilidad (Maximizan S/N)
print(f"La combinación de niveles que minimiza la variabilidad (maximizando S/N) es:")

## La combinación de niveles que minimiza la variabilidad (maximizando S/N) es:
for factor, nivel in niveles_optimos.items():
    print(f"Factor {factor}: Nivel {nivel}")

```

```

## Factor A: Nivel 2
## Factor B: Nivel 2
## Factor C: Nivel 3
## Factor D: Nivel 1

# Análisis de medias para la respuesta (X_barra)
# El objetivo es MAXIMIZAR X_barra.

 analisis_media = {}
 niveles_optimos = {}

for factor in ['A', 'B', 'C', 'D']:
    # Agrupar las medias de respuesta por nivel del factor y calcular el promedio.
    medias_por_nivel = niveles.groupby(factor)['Media'].mean().reset_index()
    analisis_media[factor] = medias_por_nivel

    # Encontrar el nivel que tiene la Media más alta.
    mejor_nivel_row = medias_por_nivel.loc[medias_por_nivel['Media'].idxmax()]
    nivel_optimo = int(mejor_nivel_row[factor])
    media_max = round(mejor_nivel_row['Media'], 3)
    niveles_optimos[factor] = nivel_optimo

    #print(f"Factor {factor}:")
    #print(medias_por_nivel.round(3))
    #print(f" -> Nivel óptimo para Media: {nivel_optimo} (Media promedio: {media_max})")

# Niveles que Maximizan la Media de la Respuesta"
print(f"La combinación de niveles que maximiza la media de la respuesta es:")

## La combinación de niveles que maximiza la media de la respuesta es:
for factor, nivel in niveles_optimos.items():
    print(f"Factor {factor}: Nivel {nivel}")

## Factor A: Nivel 2
## Factor B: Nivel 2
## Factor C: Nivel 3
## Factor D: Nivel 1

```

Boxplot para la media

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'A': [1, 1, 1, 2, 2, 2, 3, 3, 3],
    'B': [1, 2, 3, 1, 2, 3, 1, 2, 3],
    'C': [1, 2, 3, 2, 3, 1, 1, 3, 2],
    'D': [1, 2, 3, 3, 1, 2, 2, 3, 1],
    'media': [
        17.525, 19.475, 19.025, 20.125, 22.825, 19.225, 19.8875, 18.3375, 21.2
    ],
    'desviacion': [

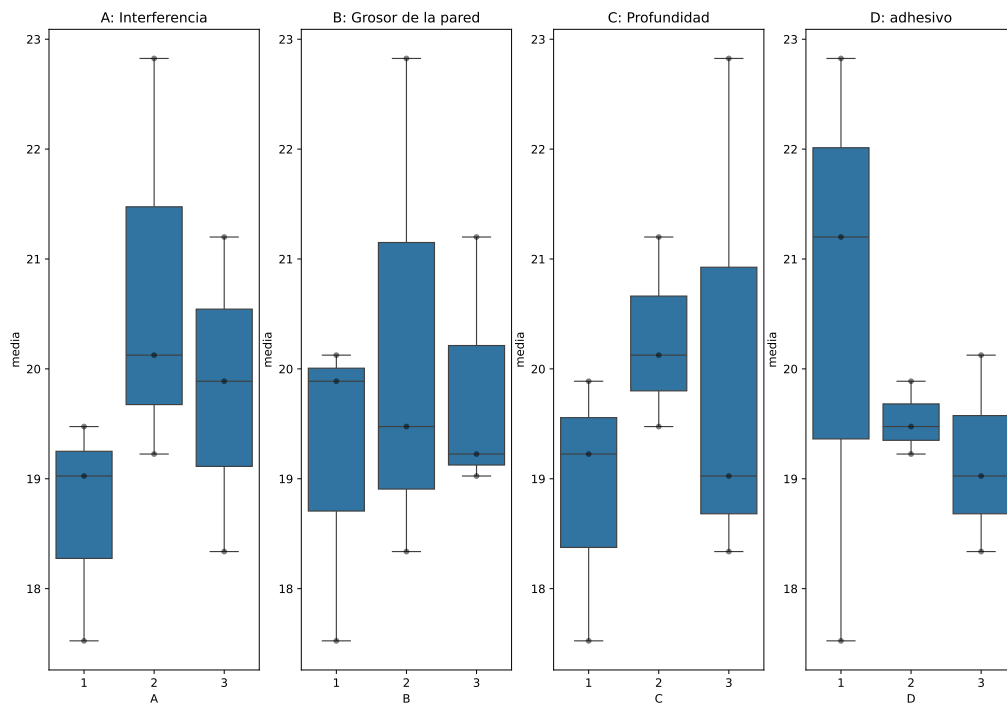
```

```

3.613, 2.907, 2.883, 2.598, 3.428, 3.380, 3.050, 3.775, 3.948
],
'mas_grande_mejor': [
24.025, 25.522, 25.335, 25.904, 26.908, 25.326, 25.719, 24.832, 26.152
]
}

datos = pd.DataFrame(data)
fig, axs = plt.subplots(1, 4, figsize=(15, 10))
axs[0].set_title('A: Interferencia')
sns.boxplot(x="A", y="media", data=datos, ax=axs[0])
sns.swarmplot( x="A", y="media", data=datos, color='black', alpha = 0.5, ax=axs[0])
axs[1].set_title('B: Grosor de la pared')
sns.boxplot(x="B", y="media", data=datos, ax=axs[1])
sns.swarmplot( x="B", y="media", data=datos, color='black', alpha = 0.5, ax=axs[1])
axs[2].set_title('C: Profundidad')
sns.boxplot(x="C", y="media", data=datos, ax=axs[2])
sns.swarmplot( x="C", y="media", data=datos, color='black', alpha = 0.5, ax=axs[2])
axs[3].set_title('D: adhesivo')
sns.boxplot(x="D", y="media", data=datos, ax=axs[3])
sns.swarmplot( x="D", y="media", data=datos, color='black', alpha = 0.5, ax=axs[3])

```



BoxPlot para la señal - ruido

```

fig, axs = plt.subplots(1, 4, figsize=(15, 10))
axs[0].set_title('A: Interferencia')

```

```

sns.boxplot(x="A", y="mas_grande_mejor", data=datos, ax=axes[0])
sns.swarmplot(x="A", y="mas_grande_mejor", data=datos, color='black',
              alpha = 0.5, ax=axes[0])

axes[1].set_title('B: Grosor de la pared')
sns.boxplot(x="B", y="mas_grande_mejor", data=datos, ax=axes[1])
sns.swarmplot(x="B", y="mas_grande_mejor", data=datos, color='black',
              alpha = 0.5, ax=axes[1])

axes[2].set_title('C: Profundidad')
sns.boxplot(x="C", y="mas_grande_mejor", data=datos, ax=axes[2])
sns.swarmplot(x="C", y="mas_grande_mejor", data=datos, color='black',
              alpha = 0.5, ax=axes[2])

axes[3].set_title('D: adhesivo')
sns.boxplot(x="D", y="mas_grande_mejor", data=datos, ax=axes[3])
sns.swarmplot(x="D", y="mas_grande_mejor", data=datos, color='black',
              alpha = 0.5, ax=axes[3]);

```

